Cost controls — spend caps, monitoring & incident response

The consolidated cost-control runbook for a deployed Claude apps gateway: how spend enforcement works, how to set and change caps, how to watch spend, what a capped developer experiences, and what to do when the fail-closed spend store takes the fleet down. This document supersedes the spend-caps section of `om-runbooks.md` as the authoritative cost doc; alarm response in general stays in that document's §9, secret handling in §7.

Every command uses the repo's scripts and `deploy.env` variables — never hardcoded org values. Run operator commands from a host with `deploy.env` filled in and AWS credentials for the deployment region (scripts source `scripts/common.sh`, which loads `deploy.env`).

---

1. Overview — how spend enforcement works

The master switch is configuration; the caps are data.

- **Master switch:** stack `02-gateway.yaml` renders the gateway's `admin:` block. The gateway runs spend enforcement **only** when `admin:` is configured — without it, even `enforcement.fail_closed_on_error` is inert. The stack also mints two API keys into Secrets Manager (`${NAME_PREFIX}/spend-admin-write-key`, `${NAME_PREFIX}/spend-admin-read-key`; `GenerateSecretString`, CMK-encrypted) and sets the retention knobs (`audit_retention_days: 365`, `spend_retention_months: 13`, `identity_retention_days: 90`).
- **Caps are DATA, not CloudFormation:** rows in the gateway's `spend_limits` table, written through `POST /v1/organizations/spend_limits`. **No cap rows = no enforcement**, so the stack is safe to deploy before any budget decisions exist, and cap changes never require a stack update.
- **Two metering paths, two authorities:**
- *Enforcement path:* client → gateway → **Postgres**. The gateway meters usage into the `spend` table as **aggregate cents per principal per period** (derived from tokens via the model rate table). This is what caps are checked against. Postgres does **not** hold per-request token counts.
- *Analytics path:* client → localhost ADOT **sidecar** → **AMP**. The `claude_code_*` metrics (`claude_code_cost_usage`, `claude_code_token_usage`, …) carry the per-request token/cost breakdown with `team` / `cost_center` / `user_groups` / `session_id` labels. This is what Grafana reads. AMP is authoritative for analytics only — an AMP outage never affects enforcement, and vice versa.
- **Scopes and precedence:** caps exist per **user** (`sub` or email), per **Okta group** (`rbac_group`), and **org-wide**. A per-user cap wins over group caps. When a user matches several group caps they combine per `SPEND_GROUP_LIMIT_MODE` (`min`, the default, takes the most restrictive — adding someone to a group can only tighten their cap; `max` takes the most permissive). Currency is USD only, enforced by the gateway.
- **Okta prerequisite for group caps:** per-group caps resolve against the **Okta groups claim**, which must actually be present in the token (the `groups` scope is requested unconditionally, but the claim itself is an org-side Okta app setting — see `../requests/okta-request-email.md`). Per-user and org-wide caps key on `sub` and keep working if the claim is missing; group caps then **silently match nothing**. Check `principal_emails.groups` via `scripts/diagnostics/dump-usage.sh` (§3.3) if group caps appear to have no effect.

---

2. Setting and changing caps

Trigger / Frequency: onboarding a team or user, a budget change, or a spend alert.

2.1 Preferred path — the portal admin page (individual identity)

With the download portal deployed (stack 04) and `PORTAL_ADMIN_GROUP` (04) plus `SPEND_ADMIN_GROUPS` (02) set to the same Okta group, members manage caps at `https://<GATEWAY_FQDN>/portal/admin` **as themselves**: the page walks the gateway's device-flow sign-in once per session, and every list/set/clear call rides the admin's own gateway token. The gateway re-checks group membership on each call and `admin_audit` records the individual actor (`oidc:<sub>`) — **no WRITE key is stored anywhere in the portal**. (The portal task does inject the READ-ONLY key for the usage read paths — §3.4 — but mutations always ride a per-admin bearer.)

Drift symptom: the page reports *"The gateway refused: your account is not in its spend-admin groups"* → `SPEND_ADMIN_GROUPS` and `PORTAL_ADMIN_GROUP` have diverged; re-align them and re-run the affected deploy script. An empty `SPEND_ADMIN_GROUPS` disables bearer-token admin entirely (only the generated keys work); an empty `PORTAL_ADMIN_GROUP` hides the admin page.

2.2 Break-glass path — `scripts/set-spend-limit.sh` (shared keys)

Works with nothing but `deploy.env` (`GATEWAY_FQDN`, `NAME_PREFIX`) and IAM access to the two key secrets — no portal, no Okta session:

```
scripts/set-spend-limit.sh --scope user       --id <okta-sub-or-email> --amount 50
scripts/set-spend-limit.sh --scope rbac_group --id <okta-group-name>   --amount 2500
scripts/set-spend-limit.sh --scope organization                        --amount 10000
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --clear   # remove a cap
scripts/set-spend-limit.sh --list                                          # review all caps
```

Mechanics that matter:

- `--amount` is **dollars** (`50` or `50.00`); the API takes whole **cents as a string** and the script converts exactly — no float rounding. `--period` is `daily` | `weekly` | `monthly` (default `monthly`).
- **Key hygiene:** the script pulls `${NAME_PREFIX}/spend-admin-write-key` (mutations) or `${NAME_PREFIX}/spend-admin-read-key` (`--list`) from Secrets Manager and hands the value to curl via a mode-600 **header file** — the key never appears on a command line (`ps` / `/proc` leak). Keep that property in any ad-hoc curl you write; better, don't write one.
- **TLS:** verification is against the system store **plus** `GATEWAY_CA_BUNDLE` and `EXTRA_CA_CERT_PATH` combined, so the script works both on a direct path (internal-PKI ALB cert) and behind TLS inspection. Never `-k`; on persistent failure the script prints the exact `openssl` command to compare the presented issuer against your bundle.

- Changes are **data-effective**: no stack update, image build, or service roll is needed for a cap to change. Always finish with `--list` to confirm what the gateway now holds.

---

3. Monitoring spend

3.1 Grafana — "Claude Code — Usage & Cost" dashboard

Provisioned from `docker/grafana/dashboards/claude-code-usage.json`; every panel honors the `Team`, `Cost center`, and `Okta group` variables (populated from the `team` / `cost_center` / `user_groups` metric labels) except **Active users (24h)**, which is deliberately org-wide and fixed-window.

The time-series live in two sections, one metric family per section pair: **"Cumulative (selected range)"** — running totals where every session seen in the range holds its final contribution to the right edge (a client going quiet does NOT drop off the graph; the right edge matches the stat tiles) — and **"Burn rate (trailing 1h)"** — activity in the trailing hour, which drains to zero within an hour of a session ending, by design. Read trends and "who is driving cost" from the cumulative section; read "who is spending right now" from the burn-rate section.

Panel	What it answers
Cost / **Tokens** / **Sessions** / **Active users** (stat row)	Totals for the selected range; sessions/active-users counted by distinct `session_id` / `user_email` **with spend activity** (`claude_code_cost_usage` samples)
Cost by team / **by cost center** / **by Okta group** (both sections)	Spend split along each org dimension — cumulative for trends, burn rate for right-now
Tokens by model (both sections)	Model mix across the three configured models (Opus vs Sonnet 5 vs the Sonnet 4.5 small/fast tier) — a cheap lever when cost spikes
Tokens by type (both sections)	input / output / cache split: cache effectiveness and prompt-heavy workloads
Top users by cost (selected range)	`topk(15)` table by `user_email` with team/cost-center — the candidates for a per-user cap
Lines of code changed / **Commits created** (both sections)	Output-side context so cost is read against delivered work

Query note: everything is **window functions**, not `increase()` — reworked after the `session.id` label fix (§6). Burn-rate panels are `max_over_time - min_over_time` over the trailing hour; the cumulative panels, tiles, and top-users table compute each session's in-range rise (counter peak minus its value at the range start when it was already running, else the full counter — so single-sample sessions count). Exact expressions and their accounting caveats: `troubleshooting.md`, dashboard section. Empty `Okta group` dropdown → the groups claim is not landing; see §1's prerequisite and §3.3.

3.2 Direct AMP queries — `scripts/diagnostics/amp-query.py`

SigV4-signed report against the AMP workspace (uses botocore's full credential chain). Env-driven: `OBSERVABILITY_AMP_ENDPOINT` (persisted into `deploy.env` by stack 03), `AWS_REGION`, and `AMP_QUERY_WINDOW_HOURS` (default 48 — client metrics are bursty; short windows miss them).

```
. scripts/deploy.env
python3 scripts/diagnostics/amp-query.py
```

It reports which `claude_code_*` metric names are stored, the `otelcol_*` heartbeat series, and — when client metrics are missing — walks the collector's own pipeline counters to a verdict (accepted vs refused vs **failed translations**, the silent-drop counter). Its 403 hints are load-bearing: `SignatureDoesNotMatch` is an encoding regression, not IAM; a plain 403 on a CMK-encrypted workspace usually means the **caller** lacks `kms:Decrypt` (`kms:ViaService=aps.<region>.amazonaws.com`).

3.3 Postgres ground truth — `scripts/diagnostics/dump-usage.sh`

Read-only dump of what the gateway has actually persisted, over the same connection path the gateway uses (`${NAME_PREFIX}/db-app-user` secret + RDS CA, verify-full). Run from an in-VPC host or bastion whose ENI carries the DB client SG (stack 01 output `DBClientSecurityGroupId`) and with IAM to read the app-user secret; `pg8000` installs offline from `docker/db-admin/vendor`. `DUMP_LIMIT` caps rows per table (default 50).

Table	Contents	Retention
`spend`	aggregate cents per principal per period — the enforcement ledger	13 months
`spend_limits`	the configured caps (what `--list` shows)	live data
`principal_emails`	principal → email/name + the resolved **Okta groups claim**	90 days
`admin_audit`	every admin API mutation, with actor attribution	365 days

Interpretation: empty `spend` → the gateway is metering nothing (no inference has flowed, or metering is broken); NULL/empty `principal_emails.groups` → group caps match nothing and the Grafana Okta group filter is empty. **Raw per-request token/cost detail lives only in AMP** — Postgres holds aggregates; use §3.1/§3.2 for the breakdown and `scripts/diagnostics/diagnose-telemetry.sh` for pipeline health.

3.4 Portal read paths — self-view and the all-users table

The download portal carries two **read-only** spend views, both backed by the gateway's effective-limits API — `GET /v1/organizations/spend_limits/effective`, which returns one row per user per period with the **effective** cap (after user/group/org precedence), the scope it came from, and `period_to_date_spend` (the same gateway-metered cents the enforcement path uses; may be fractional).

- `/portal/me` — every signed-in portal user sees their **own** caps and period-to-date spend (per period, with cap source and percent used). The portal makes this call server-side with the injected `${NAME_PREFIX}/spend-admin-read-key` (`SPEND_READ_KEY`, imported from 02's export), pinned to the session's own Okta `sub`. If the key is unset the page reports the feature disabled; enabling it on a deployment that predates the export = 02 re-run (creates the export), then 04.
- `/portal/admin/users` — a paged, searchable all-users table (name, email, sub, groups, cap, period, spend to date, % used, cap source) for `PORTAL_ADMIN_GROUP` members. It deliberately does **not** use the read key: calls carry the admin's own device-flow bearer, so the gateway re-checks group membership per call and attribution stays individual. Gateway-side constraints surfaced in the UI: sort-by-spend requires selecting a single period, and paging is forward-only.

Neither path can mutate anything: the read key is list/read-scoped and the write key never reaches stack 04. The full posture discussion is in `../ato/security-assessment-2026-07.md`.

---

4. What a capped user experiences — and lifting a cap fast

A developer over their cap gets **HTTP 429**, error type `billing_error`, message *"spend limit reached — "*, with `x-should-retry: false`. It is **not** a transient error: the client will not retry around it, and waiting does not help until the period rolls over or the cap changes. Set `SPEND_BLOCKED_MESSAGE` in `deploy.env` to org- specific routing text ("contact for an increase") — it is the only self-service breadcrumb the developer sees.

Confirm it is a cap (60 seconds):

```
scripts/set-spend-limit.sh --list        # is there a cap matching this user / their groups / org?
scripts/diagnostics/dump-usage.sh        # spend row for the principal vs the cap amount
```

One user capped with caps listed and a matching `spend` row → working as designed. **Everyone** capped at once → this is probably not a cap at all; go straight to §5.

Lift or raise:

```
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --amount <new-dollars>
# or remove entirely:
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --clear
```

Data-effective immediately on the gateway side (no redeploy); have the user retry, and remember `min` mode means a generous per-group cap cannot loosen a tighter one — a per-user cap is the reliable override, since per-user always wins.

---

5. INCIDENT RUNBOOK — fail-closed spend-store outage

Trigger / Frequency: fleet-wide 429s reported by developers, or DB alarms. **No dedicated CloudWatch alarm watches this condition** — detection today is user reports plus the DB-side alarms in `om-runbooks.md` §9.

Why this exists: `enforcement.fail_closed_on_error: true` is a hardcoded operator decision in `cloudformation/02-gateway.yaml`'s rendered gateway config. If the spend store is unreachable or errors, the gateway returns 429 **for every request** rather than allow uncapped spend. This is a deliberate availability trade: **an RDS/spend-store outage halts all inference fleet-wide, not just cost tracking.**

Symptoms:

- Sudden fleet-wide 429s ("spend limit reached" / "spend limit unavailable") affecting **every** user simultaneously — including users with no cap set.
- `spend check failed` / `store_error` entries in the gateway log group.
- Correlated RDS trouble: `${NAME_PREFIX}-db-rotation-errors` alarm, RDS instance events, or a recent DB credential rotation.

Triage (in order):

1. Confirm scope: one user capped = §4, not an incident. All users = here.
2. Check the gateway log group for `spend check failed` / `store_error`.
3. Check RDS: instance status/events in the console, storage/connection exhaustion, and the `${NAME_PREFIX}-db-rotation-errors` alarm — a botched app-credential rotation looks exactly like a store outage to the gateway (recovery in `om-runbooks.md` §3).
4. Check network path: DB SG membership unchanged, gateway tasks healthy.

Recovery:

1. **Fix the database** — that is the real fix; enforcement recovers on its own once the store answers.
2. **Break-glass (only if inference must resume before the store is fixed):** flip `fail_closed_on_error: true` → `false` in the `enforcement:` block of `cloudformation/02-gateway.yaml` (it is a template literal, not a parameter) and re-run `scripts/deploy-gateway.sh`. Spend is now **unmetered and uncapped** while the store is down.

3. **Obligation:** the flip is temporary by definition. Track it as an open incident action; once the store is healthy, revert the template edit, re-run `scripts/deploy-gateway.sh`, and confirm enforcement is back with `scripts/set-spend-limit.sh --list` plus a capped-user probe. Do not commit the flipped value — the repo posture is fail-closed.

Verification after recovery: `--list` succeeds; `spend` rows advance again (`scripts/diagnostics/dump-usage.sh`); no `store_error` in fresh gateway logs.

---

6. Gaps & recommendations (honest)

- **No AWS-account-level budget alarm exists.** Nothing in `cloudformation/` creates an `AWS::Budgets::Budget` or Cost Explorer–based alarm; the alarms that do exist (§9 of `om-runbooks.md`, `MissingTelemetryAlarm` in `cloudformation/03-observability.yaml`) watch **pipeline health**, not dollars. Today's defenses are app-level caps only — which enforce nothing until someone writes cap rows, and which a fail-closed flip (§5.2) temporarily disables. **Recommendation:** add an AWS Budgets alert (or Cost Explorer anomaly detection) on this account's **Bedrock** spend as an independent backstop that catches runaway cost even when app-level enforcement is off, misconfigured, or bypassed. Deliberately left as a recommendation — budgets are org policy, and GovCloud billing-data flows vary by agreement; decide placement with the finance owner before adding CFN.
- **The org-wide cap is the closest existing backstop.** Until a budget alarm exists, consider a generous `--scope organization` cap sized well above expected monthly spend, so a metering-side runaway hits *something*.
- **Dashboard history predating the `session.id` fix is unreliable.** The sidecar once deleted `session.id` for cardinality; concurrent sessions from one user then interleaved onto a single series as a sawtooth and `increase()` drastically inflated dashboard spend. `session.id` is now kept — each session is its own monotonic series — and the dashboard panels use window functions accordingly (§3.1). Postgres `spend` was and remains the enforcement ledger, so use it, not dashboard history, for any retrospective accounting.

---

7. Audit trail

Every spend-limit mutation lands in the gateway's `admin_audit` table (365-day retention), attributed to the acting identity:

- `oidc:<sub>` — an individual admin acting through the portal admin page (or any bearer-token call allowed by `SPEND_ADMIN_GROUPS`).
- **key** `id` (`deploy-write` / `deploy-read`) — the break-glass CLI's shared keys. This is why the portal path is preferred: shared-key entries identify a credential, not a person.

The portal admin page shows this trail read-only and additionally writes `event: portal_admin` lines (connects, actions, denials) to the portal's own audit log group. Inspect the table directly with `scripts/diagnostics/dump-usage.sh` (§3.3).

Who is `oidc:<sub>`? — the Okta email, three ways. The `admin_audit` table belongs to the gateway binary, so its schema cannot grow an email column; the email is captured and joined alongside it instead:

- **Portal audit page** (`/portal/admin/audit`): an **Email** column. When an admin connects a gateway session, the portal holds both halves of the identity — the portal session's Okta email and the gateway token's `sub` (the exact value `admin_audit` will record) — and persists the pairing as one small JSON object per sub under the reserved `identity/principal-emails/` prefix of the portal artifacts bucket (CMK-encrypted; the task role may write only that prefix). The audit page joins actors against this map. Actors who have never connected through the portal since the map was introduced — including the break-glass CLI keys — show a dash.
- **Portal audit log group**: `event: portal_admin` lines carry both `user_email` and `gateway_actor` (`oidc:<sub>`), so a gateway row can be tied to an emailed portal line even without the map.
- `dump-usage.sh` **(§3.3)**: the `admin_audit` dump LEFT JOINs the gateway's own `principal_emails` table (`principal` = the sub the gateway resolved at login) and prints an email per row — this covers *all* users the gateway has ever identified, independent of the portal map.

Key rotation: both admin keys are `GenerateSecretString` secrets, injected as ECS `Secrets` and read only at container start. Rotate by writing a new value with the file-based no-argv pattern and then **forcing a new deployment** (`aws ecs update-service --cluster <cluster> --service ${NAME_PREFIX}-gateway --force-new-deployment`) — the same procedure as the gateway JWT secret in `om-runbooks.md` §7. A plain `deploy-gateway.sh` re-run is NOT enough: the secret write happens outside CloudFormation, so the stack update is an empty changeset and running tasks keep the old keys. Rotating the write key invalidates any operator's cached copy but not portal admins, who never use it. Rotating the **read** key: the portal task injects it too (`SPEND_READ_KEY`), so also force-roll `${NAME_PREFIX}-portal`, or `/portal/me` fails with a stale-key 401 until the next portal deployment.